
CSE 151B/251B Final Report

Sergi Marsol Torrent Sritha Kondragunta Lillian Liu Christina Xu

Department of Computer Science and Engineering

University of California, San Diego

smarsoltorrent@ucsd.edu, skondragunta@ucsd.edu, ly1008@ucsd.edu, chx029@ucsd.edu

Abstract

We study how optimization and architectural refinements affect decoder-only Transformer performance under a strict 100M parameter budget. Starting from the nanoGPT baseline, we improve training stability and parameter efficiency using hybrid Muon/AdamW optimization, RoPE positional embeddings, RMSNorm, QK-normalization, SwiGLU feedforward layers, gradient accumulation, and cosine learning-rate scheduling.

Through controlled ablations and scaling experiments on FineWeb-Edu, we find that optimization design has a larger impact on validation perplexity than parameter scaling alone. Our final 98.9M parameter model achieves validation perplexity 24.99, substantially outperforming the original nanoGPT baseline while remaining within project constraints.

Code and experiments are available at: <https://github.com/christinaxu10/251b-nanogpt>.

1 Introduction

Problem Definition. This project trains a GPT-style decoder-only language model that minimizes validation perplexity under a strict 100M parameter budget. Given an input sequence $(x_1, \dots, x_t)$, the model predicts the next-token distribution $P(x_{t+1} \mid x_{\leq t})$ using autoregressive cross-entropy training.

Starting from the nanoGPT baseline, we study how optimization, architectural modifications, and training techniques affect convergence and perplexity under fixed compute and memory constraints.

Problem Significance. Improving language-model performance without increasing parameter count reduces training cost, inference latency, and memory usage. Constrained-scale language modeling also provides a useful setting for studying which optimization and architectural choices most effectively improve Transformer training.

Technical Challenge. The main challenge is balancing model capacity, optimization stability, and computational efficiency within a strict parameter budget. Scaling model depth or width improves representational power but increases instability and memory usage, while optimizer and architectural changes are sensitive to learning rate, batch size, and normalization strategy.

Efficient experimentation is also difficult because full training runs are expensive, requiring many decisions to be evaluated on reduced datasets or shortened schedules before scaling.

Contributions. Starting from nanoGPT [2], we incorporated optimization, architectural, and engineering improvements inspired by modern decoder-only Transformers.

Our main contributions are:

- Controlled ablations studying model scale, learning rate, optimizer configuration, and effective batch size, identifying optimization stability as a key bottleneck.
- Implementation and tuning of hybrid Muon/AdamW optimization [1], improving convergence over the AdamW baseline.
- Incorporation of RoPE [5], RMSNorm, QK-normalization, and SwiGLU layers [4], consistently reducing validation perplexity.
- Development of an efficient training pipeline using gradient accumulation, mixed precision, Flash Attention, and cosine learning-rate scheduling.
- A final 98.9M parameter model achieving validation perplexity 24.99 within project constraints.

2 Related Work

GPT-style Autoregressive Language Modeling. GPT introduced decoder-only autoregressive language modeling using causal self-attention and next-token prediction [3]. Our work follows this framework while focusing on improving optimization efficiency and parameter utilization under a strict 100M parameter budget.

nanoGPT. We use Andrej Karpathy’s nanoGPT implementation [2] as the primary baseline and training framework. Its minimal GPT implementation makes it well suited for controlled experimentation. Our work extends nanoGPT through optimizer modifications, architectural upgrades, improved normalization, alternative positional embeddings, gated feedforward layers, and revised learning-rate schedules.

Muon Optimizer. To improve optimization stability, we build on the Muon optimizer [1], which applies geometry-aware updates to matrix-valued parameters. We adopt the hybrid strategy proposed in this work, applying Muon to Transformer weight matrices while retaining AdamW for biases and normalization parameters. Our experiments showed that optimization changes produced some of the largest improvements in validation perplexity.

Modern Transformer Architectural Modifications. Our final model incorporates several modern Transformer refinements. We replace learned positional embeddings with RoPE [5], replace GELU feedforward layers with SwiGLU activations [4], and incorporate RMSNorm and QK-normalization for improved training stability. Together, these modifications consistently improved convergence and final perplexity relative to the original nanoGPT baseline.

3 Methods

Problem Setting. The objective of this project is to train a decoder-only autoregressive language model that minimizes validation perplexity under a strict budget of 100M parameters.

Given a tokenized sequence

$$x = (x_1, x_2, \dots, x_T),$$

the model predicts the conditional probability of the next token:

$$P(x_{t+1} \mid x_{\leq t}).$$

Training minimizes the autoregressive negative log-likelihood objective:

$$\mathcal{L}(\theta) = -\frac{1}{N} \sum_{i=1}^N \sum_{t=1}^{T_i} \log P_{\theta}(x_{i,t} \mid x_{i,<t}),$$

where θ denotes the model parameters and N is the number of training sequences.

Model quality is evaluated using perplexity:

$$\text{PPL} = \exp \left(\frac{1}{N} \sum_{i=1}^N -\log P(t_i \mid t_{<i}) \right),$$

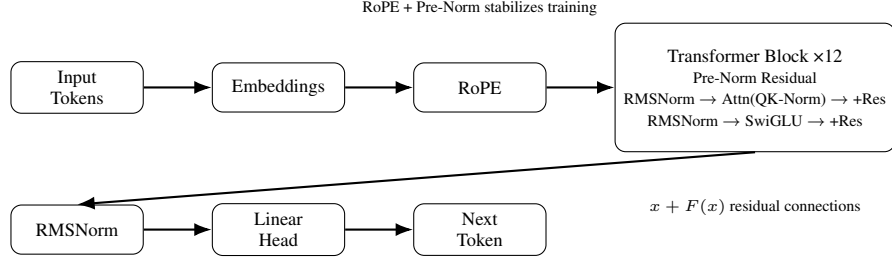


Figure 1: Final decoder-only Transformer architecture. Relative to the original nanoGPT baseline, the model incorporates RoPE positional embeddings, RMSNorm, QK-normalization, SwiGLU feedforward layers, gradient accumulation, and hybrid Muon/AdamW optimization.

which measures uncertainty when predicting unseen text. Inputs to the model consist of fixed-length GPT-2 token sequences with vocabulary size 50,257. Targets are formed by shifting each sequence by one token for autoregressive next-token prediction.

Training data is derived from the FineWeb-Edu corpus. During development, we additionally created reduced 10%, 20%, and 50% subsets for faster architectural exploration and hyperparameter tuning. All experiments used a fixed held-out validation shard for consistent comparison across runs.

Idea Summary. Rather than relying primarily on model scaling, our approach focuses on improving optimization efficiency and parameter utilization within the fixed 100M parameter budget. Starting from the nanoGPT baseline [2], we incorporated optimization and architectural techniques inspired by modern decoder-only Transformer designs.

Our final approach combines several modifications. We replaced learned positional embeddings with Rotary Positional Embeddings (RoPE) [5], replaced LayerNorm and GELU feedforward layers with RMSNorm and SwiGLU activations [4], and incorporated QK-normalization together with warmup-stable-decay scheduling. Finally, we adopted a hybrid Muon/AdamW optimization strategy based on the Muon optimizer [1].

In practice, optimization quality and training stability became increasingly important as models approached the 100M parameter limit. Promising configurations were progressively validated on larger dataset subsets and longer training schedules before inclusion in the final model.

Method Description. Our final model is a decoder-only Transformer containing 12 decoder blocks, embedding dimension $d = 672$, 12 attention heads, and context length 1024, for a total of approximately 98.9M parameters.

The architecture follows the GPT design with several modifications relative to the original nanoGPT implementation. Learned positional embeddings were replaced with Rotary Positional Embeddings (RoPE), which encode positional information directly within attention computations.

We additionally replaced LayerNorm with RMSNorm and incorporated QK-normalization within attention computations to improve optimization stability. The standard GELU feedforward network was replaced with SwiGLU feedforward layers motivated by recent Transformer architectures.

Optimization used a hybrid Muon/AdamW configuration. Muon updates were applied primarily to matrix-valued Transformer parameters, while AdamW optimized remaining parameters such as normalization weights and biases. Training additionally employed gradient accumulation, mixed-precision computation (bfloat16), gradient clipping, and cosine learning-rate decay with linear warmup.

Our experimentation process emphasized incremental scaling and optimization refinement. Early exploratory experiments were conducted on reduced FineWeb-Edu subsets using smaller models and shorter schedules before progressively scaling successful configurations toward the final 98.9M parameter system.

Figure 1 summarizes the final architecture and the primary modifications introduced relative to the original nanoGPT baseline.

4 Experiments

In this section, we evaluate how optimization, model scaling, and architectural refinements affect decoder-only Transformer performance under a fixed 100M parameter budget.

All experiments use the same held-out FineWeb-Edu validation shard and report validation perplexity as the primary evaluation metric.

4.1 Baselines

We compare progressively stronger baselines to evaluate the impact of optimization and scaling improvements relative to the original nanoGPT implementation. We compare the following baselines:

- **Baseline A (nanoGPT baseline).** The original nanoGPT implementation [2], consisting of a decoder-only Transformer with learned positional embeddings, LayerNorm, GELU feedforward layers, and AdamW optimization.
- **Baseline B (scaled AdamW baseline).** A larger nanoGPT model trained with improved learning-rate scheduling and larger effective batch sizes while retaining AdamW optimization.
- **Baseline C (Muon baseline).** A scaled Transformer trained using hybrid Muon/AdamW optimization [1], which served as the foundation for later refinement experiments.
- **Final model.** Our best-performing configuration combining Muon optimization, RoPE positional embeddings, RMSNorm, QK-normalization, SwiGLU feedforward layers, gradient accumulation, and stabilized learning-rate scheduling.

Table 1 summarizes representative configurations explored throughout the project.

Table 1: Representative model configurations explored throughout the project.				
Model	Layers / Dim	Optimizer	Params	Validation PPL
nanoGPT baseline	6L / 384	AdamW	30.3M	58.26
Larger nanoGPT baseline	8L / 512	AdamW	51.5M	45.53
Early Muon baseline	8L / 512	Muon + AdamW	51.5M	40.88
Scaled Muon model	10L / 640	Muon + AdamW	82.1M	29.42
Final model	12L / 672	Muon + AdamW	98.9M	24.99

4.2 Evaluation

We evaluate all models using validation perplexity (PPL), computed as:

$$\text{PPL} = \exp \left(\frac{1}{N} \sum_{i=1}^N -\log P(t_i | t_{<i}) \right),$$

Lower perplexity indicates better next-token prediction and generalization to unseen text. All experiments use the same held-out FineWeb-Edu validation shard to ensure consistent comparison across runs. We also analyze validation-loss trajectories to assess convergence behavior, optimization stability, and scaling efficiency across configurations.

4.3 Implementation Details

All models were implemented in PyTorch and trained on NVIDIA A100 GPUs using the UCSD DataHub environment. After encountering DSMLP disk storage capacity issues, we transitioned to Google Colab, still training on NVIDIA A100 GPUs. Due to limited compute, we followed a staged approach: early runs used reduced FineWeb-Edu subsets, with promising configurations later scaled to larger datasets and longer training schedules.

Initial experiments included smaller 4-layer and 6-layer Transformers trained for short schedules to validate the training pipeline and assess optimization stability. These runs often yielded very high

perplexities (e.g., > 400 PPL), highlighting the sensitivity of small-scale language-model training to optimizer settings, batch size, and learning rate.

Training used mixed precision (bfloat16) and PyTorch scaled dot-product attention when available. We also applied gradient clipping, cosine learning-rate decay with linear warmup, and gradient accumulation to improve stability in longer runs.

The best model was trained for 35,000 steps with gradient accumulation, reaching an effective throughput of 98k tokens per step, for a total of 3.4B tokens. Training took approximately 5 hours and 15 minutes of wall-clock time.

Hyperparameter tuning focused on learning rate, effective batch size, model width, optimizer configuration, and stabilization methods. Rather than exhaustive grid search, we used iterative targeted sweeps guided by validation perplexity and convergence trends.

Table 2 summarizes the final model configuration.

Table 2: Final configuration used for the best-performing model.

Component	Final Setting
Layers	12
Hidden dimension	672
Attention heads	12
Context length	1024
Optimizer	Muon + AdamW
Precision	bfloat16
Learning-rate schedule	Warmup-Stable-Decay
Gradient accumulation steps	12
Training steps	35,000
Approx. tokens processed	3.4B
Dataset	fineweb-10pct-99
Total parameters	98.9M

Figure 2 summarizes the overall experimentation workflow used throughout the project.

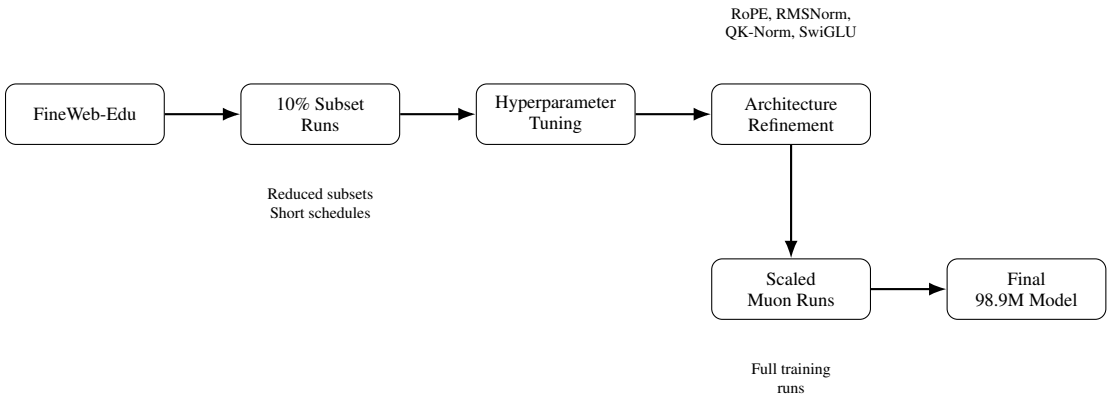


Figure 2: Overview of the experimentation workflow. Early-stage hyperparameter and architectural exploration was performed on reduced FineWeb-Edu subsets before progressively scaling promising configurations to larger datasets and longer training schedules.

4.4 Results

Early-stage experiments during the milestone phase produced substantially worse perplexities than the final system, with some unstable small-scale Muon configurations exceeding 600 validation PPL. While not competitive, these runs highlighted optimization sensitivity and informed later improvements in gradient accumulation, learning-rate scheduling, and stabilization methods.

Optimization and Convergence Behavior. Optimization stability was critical to final performance. Early results showed that increasing model scale without corresponding optimization improvements often led to unstable convergence and limited perplexity gains.

Figure 3 compares representative validation-loss trajectories across project stages. Increasing effective batch size and improving learning-rate scheduling significantly improved convergence relative to the nanoGPT baseline. Introducing hybrid Muon/AdamW optimization further accelerated convergence and reduced final validation perplexity.

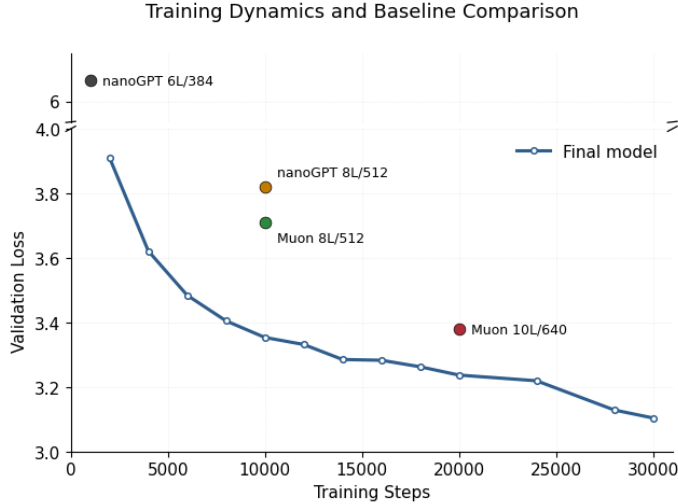


Figure 3: Validation-loss curve for the final architecture, evaluated every 200 training steps. Additional markers indicate the final validation-loss endpoints of alternative nanoGPT and Muon-based configurations. Improved optimization and larger effective batch sizes significantly improved convergence stability and final perplexity.

Final Model Performance. Table 3 summarizes representative validation perplexities across major stages of the project.

Table 3: Representative progression of validation perplexity across major stages of the project.

Configuration	Parameters	Validation PPL
4L / 256 exploratory Muon run	16.3M	632.48
6L / 384 nanoGPT baseline	30.3M	58.26
8L / 512 nanoGPT baseline	51.5M	45.53
8L / 512 + Muon	51.5M	40.88
10L / 640 Muon model	82.1M	29.42
Experiment 3	91.4M	29.66
Experiment 3.5	91.4M	28.73
Final model	98.9M	24.99

Improvements were driven more by optimization and stabilization than by parameter scaling alone. Larger effective batch sizes and Muon-based optimization notably improved convergence as model scale increased.

Further gains in later stages came from incorporating RMSNorm, QK-normalization, and stabilized learning-rate scheduling in otherwise comparable 91M-parameter setups.

Figure 4 shows validation perplexity across progressively scaled configurations.

To qualitatively assess generation quality, we sampled from the final checkpoint using temperature sampling ($T = 0.6$) with top- $k = 200$. The model still occasionally showed repetition and semantic drift, but generally produced syntactically coherent outputs and could sustain structured continuations across factual, Q&A, and literary prompts.

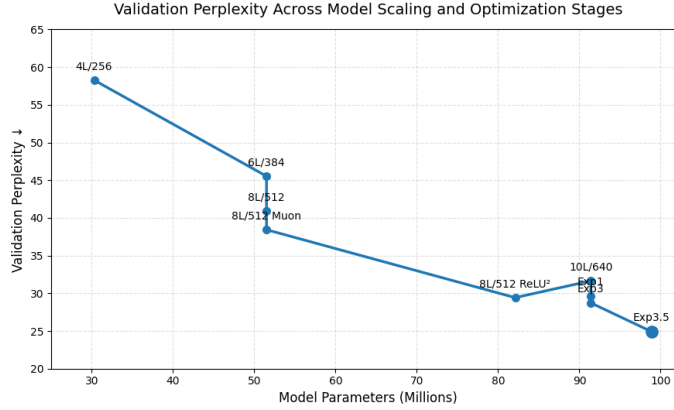


Figure 4: Validation perplexity across progressively scaled model configurations. Increasing model scale improved performance consistently, while optimization and stabilization refinements produced the largest reductions in perplexity.

Prompt: “Photosynthesis is the process by which plants”
Generated: “Photosynthesis is the process by which plants use sunlight to convert carbon dioxide into sugars and other chemical compounds...”
Prompt: “Question: How do neural networks learn?”
Generated: “Neural networks learn by doing. The neural network is a collection of information that is stored in a computer and is used to learn...”
Prompt: “from fairest creatures we desire increas”
Generated: “from fairest creatures we desire increase. The world is not so much a desert that we are called to look upon...”

Figure 5: Representative text generations from the final 98.9M parameter model using temperature sampling ($T = 0.6$) and top- $k = 200$. The model produces coherent sentence structure and adapts to multiple prompt styles, although some repetition and semantic instability remain during longer continuations.

Figure 5 shows representative generations from the final 98.9M parameter model.

4.5 Ablations

Due to limited compute, we balanced optimization and scaling experiments with exhaustive architectural ablations. In addition to our ablations, we focused on controlled experiments targeting optimization stability, training sensitivity, and incremental architectural refinements.

Hyperparameter Ablations. The Adam baseline achieves a validation perplexity of **469.81**. The best values for each ablated parameter are:

- Learning rate: 10^{-3} (PPL: 446.79)
- Embedding dimension: 640 (PPL: 412.82)
- Number of layers: 8 (PPL: 464.10)
- Number of heads: 128 (PPL: 468.06)
- Batch size: 10 (PPL: 424.39)

Learning rate has the largest impact on performance, with both too large (10^{-2} : 925.81) and too small (10^{-4} : 726.80) values leading to significantly worse perplexity. Increasing embedding dimension provides the most consistent improvement, reducing perplexity from 552.96 (128) to 412.82 (640), after which gains diminish. Increasing the number of layers yields only marginal improvements, and

the number of heads has minimal effect. Increasing batch size improves performance (469.81 $\rightarrow$ 424.39) but is limited by memory constraints.

Gradient Accumulation and Muon Optimization. We first evaluated gradient accumulation effects on training stability and validation perplexity. Small effective batch sizes led to unstable optimization and poor convergence. Increasing effective batch size substantially improved convergence and reduced perplexity, highlighting the importance of optimization dynamics in training. On top of the hybrid Muon baseline, we also tested sensitivity to Muon and AdamW learning rates. Performance improved significantly after tuning the Muon learning rate, while tuning the AdamW learning rate had minor improvement. The baseline learning rate schedule with minimal decay performed better than a cooldown scheme that cooled to a learning rate of an order smaller. These results show Muon optimization requires careful learning-rate tuning, with higher rates causing instability.

Table 4: Effect of gradient accumulation on validation perplexity.

Gradient Accumulation Steps	Validation PPL
1	632.48
4	440.96
8	101.07
12	88.68
16	81.53

Table 5: Sensitivity of validation perplexity to Muon learning rate.

Muon LR	Validation PPL
0.005	139.55
0.010	131.28
≥ 0.020	Diverged / unstable

Hybrid Muon–AdamW Parameter Split. We compared a broad parameter split, which applied Muon to all two-dimensional parameters, against a cleaner split that assigned embeddings and the tied output head to AdamW. Using the original learning rates, the clean split substantially increased validation loss at 4,500 iterations from 3.6588 to 3.8390. Increasing the AdamW and Muon learning rates closed most of this gap, but the broad split remained slightly better at both 4,500 and 9,000 iterations. We therefore retained the broad split for the final model.

Table 6: Comparison of broad and clean Muon–AdamW parameter splits at 4,500 iterations.

Experiment	Muon LR	AdamW LR	Train Loss	Val Loss	Δ Val Loss
Broad split baseline	3×10^{-3}	1×10^{-4}	3.6426	3.6588	–
Clean split, original LR	3×10^{-3}	1×10^{-4}	3.8253	3.8390	+0.1802
Clean split, higher Muon and AdamW LR	5×10^{-3}	3×10^{-4}	3.6588	3.6751	+0.0163
Clean split, highest Muon LR	7×10^{-3}	3×10^{-4}	3.6525	3.6673	+0.0085

Table 7: Longer-run comparison of broad and clean Muon–AdamW parameter splits at 9,000 iterations.

Experiment	Muon LR	AdamW LR	Train Loss	Val Loss	Δ Val Loss
Broad split baseline	3×10^{-3}	1×10^{-4}	3.4360	3.4404	–
Clean split, highest Muon LR	7×10^{-3}	3×10^{-4}	3.4526	3.4550	+0.0146

Architectural Refinements. We then evaluated stabilization and architectural refinements in larger models. In comparable 91M-parameter setups, adding RMSNorm, QK-normalization, and warmup-

stable-decay improved validation perplexity from 31.64 to 29.66. We also compared activation functions in 51M-parameter models: ReLU² achieved 38.43 PPL, while SwiGLU achieved 39.47.

Overall, results indicate optimization and stabilization choices had a larger impact on performance than individual architectural changes.

Restarted Stable–Decay Phase. As a final design choice, we extended training by restarting the learning rate at its stable maximum and applying another short decay to the minimum rate. This acted as a learning-rate restart: the higher rate allowed the model to move beyond the solution reached after the first decay, while the subsequent rapid decay refined the new solution. In a non-final model run, this reduced public-validation perplexity from 29.66 to 28.73, whereas continuing training only at the minimum learning rate produced little improvement. Applying the same strategy to the final model further reduced perplexity from 25.24 to 24.90 on the public validation set. This consistent improvement suggests that an additional stable–decay phase can provide modest gains after an otherwise complete training run.

5 Discussion

This project shows that strong optimization design and incremental architectural refinement can substantially improve language-model performance under tight compute and parameter limits. Starting from the nanoGPT baseline, we improved training stability and model design through controlled experimentation, reaching a final validation perplexity of 24.99 with a 98.9M parameter Transformer. The jump from early unstable runs (>600 PPL) to the final model highlights the importance of optimization stability at small-to-medium scale.

Achievements. We built an efficient GPT-style training pipeline for the sub-100M parameter regime, showing that optimization quality and stability mattered more than scaling alone. Validation perplexity improved from 58.26 (6L/384 baseline) to 24.99, driven by Muon/AdamW hybrid optimization, gradient accumulation, improved learning-rate scheduling, and architectural choices such as RoPE, RMSNorm, QK-normalization, and SwiGLU, forming a reproducible medium-scale training setup.

Bottlenecks and Mitigation. Compute limits constrained both scale and the number of full runs, addressed through staged training on reduced FineWeb-Edu subsets before scaling promising configurations. A major challenge was optimization instability at larger scales, mitigated using gradient accumulation, warmup scheduling, mixed precision, and Muon/AdamW optimization, enabling stable convergence of the final 98.9M model. Later work shifted from broad hyperparameter search to focused optimization and scaling, producing clearer and more reliable improvements.

Lessons Learned. Optimization choices often outweighed architectural changes, with batch size and training stability having stronger effects than most model tweaks. Focused experimentation proved more effective than broad ablations, producing clearer insights. The project also provided practical experience with Transformer training, mixed precision, optimizer design, and debugging instability. Future work would explore longer context, more systematic Muon tuning, tokenizer improvements, and controlled architectural comparisons under matched optimization settings.

Team Responsibilities. The project was completed by Sergi, Sritha, Lillian, and Christina, all contributing to experimentation, implementation, debugging, and analysis. Sergi led writing and results organization; Sritha led presentation and analysis; Lillian and Christina focused on training runs, architectural experiments, and evaluation; early experimentation and debugging were shared across the team.

LLM Usage

We used large language models (LLMs) as a supportive tool during this project in three main ways: (1) coding assistance for debugging and implementation support, (2) writing assistance for improving clarity, structure, and conciseness in the report, and (3) LaTeX assistance for formatting figures, tables, and resolving compilation issues.

All experimental design decisions, implementations, and analyses were performed by the project team. LLM outputs were used only under supervision and were reviewed before inclusion. The authors take full responsibility for all results and content presented in this report.

References

- [1] Keller Jordan. Muon optimizer. <https://kellerjordan.github.io/posts/muon/>, 2024.
- [2] Andrej Karpathy. nanogpt. <https://github.com/karpathy/nanoGPT>, 2023.
- [3] Alec Radford, Karthik Narasimhan, Tim Salimans, Ilya Sutskever, et al. Improving language understanding by generative pre-training. 2018.
- [4] Noam Shazeer. Glu variants improve transformer. *arXiv preprint arXiv:2002.05202*, 2020.
- [5] Jianlin Su, Yu Lu, Shengfeng Pan, Bo Wen, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024.